

Text-to-OverpassQL: A Natural Language Interface for Complex Geodata Querying of OpenStreetMap

Michael Staniek^{*1} Raphael Schumann^{*1} Maike Züfle^{◇1,2} Stefan Riezler^{1,3}

¹Computational Linguistics, Heidelberg University, Germany

²School of Informatics, University of Edinburgh, UK

³IWR, Heidelberg University, Germany

{staniek, rschuman, zuefle, riezler}@cl.uni-heidelberg.de

Abstract

We present Text-to-OverpassQL, a task designed to facilitate a natural language interface for querying geodata from OpenStreetMap (OSM). The Overpass Query Language (OverpassQL) allows users to formulate complex database queries and is widely adopted in the OSM ecosystem. Generating Overpass queries from natural language input serves multiple use-cases. It enables novice users to utilize OverpassQL without prior knowledge, assists experienced users with crafting advanced queries, and enables tool-augmented large language models to access information stored in the OSM database. In order to assess the performance of current sequence generation models on this task, we propose OverpassNL,¹ a dataset of 8,352 queries with corresponding natural language inputs. We further introduce task specific evaluation metrics and ground the evaluation of the Text-to-OverpassQL task by executing the queries against the OSM database. We establish strong baselines by finetuning sequence-to-sequence models and adapting large language models with in-context examples. The detailed evaluation reveals strengths and weaknesses of the considered learning strategies, laying the foundations for further research into the Text-to-OverpassQL task.

1 Introduction

The OpenStreetMap (OSM) database stores vast amounts of structured knowledge about our world. Users mainly access it via applications that render a visual map of inquired areas. A more advanced and systematic way to examine the stored information is to query the underlying geodata using

the Overpass Query Language (OverpassQL). OverpassQL is a feature-rich query language that is widely adopted in the OSM ecosystem by contributors, analysts, and applications. In order to make this empowering query language accessible via natural language, we propose the Text-to-OverpassQL task. The objective is to take a complex data request in natural language and translate it to OverpassQL in order to execute it against the OSM database. Several groups of users can benefit from such a natural language interface. Inexperienced users are spared from learning the OverpassQL syntax. Expert users can use it to draft Overpass queries and then manually refine them, saving time and mental load in comparison to writing the full query from scratch. Another use-case is to incorporate the interface as an API tool (Schick et al., 2023) for large language models (LLMs).

In this work, we introduce the three components that enable the Text-to-OverpassQL task. First, we present OverpassNL, a dataset of 8.5k natural language inputs and corresponding Overpass queries. The queries were collected from an OSM community website where they were written by OSM users to fulfill legitimate information needs. We then hired and trained students to write natural language descriptions of the queries. Second, we introduce a systematic evaluation protocol that assesses the prediction quality of a candidate system. To this end, we propose a task-specific metric that takes the similarity of the system output to Overpass queries on the levels of surface string, semantics, and syntax into account. Moreover, we ground the evaluation by executing the generated query against the OSM database and compare the returned elements with those returned by the gold query. Third, we explore several models and learning strategies to establish a base performance for the problem of generating

^{*}Equal contribution.

[◇]Work done while at Computational Linguistics, Heidelberg University.

¹<https://github.com/raphael-sch/OverpassNL>.

Natural Language Input:

Bike lanes 500 meters around the top of a hill or mountain in Troms.

Overpass Query Language:

```
1 {{geocodeArea:"Troms"}}->.searchArea;  
2 (  
3   node["natural"="peak"](area.searchArea);  
4 )->.peaks;  
5 way["highway"="cycleway"](area.searchArea)(around.peaks:500);  
6 out center;
```

Query Execution Results:

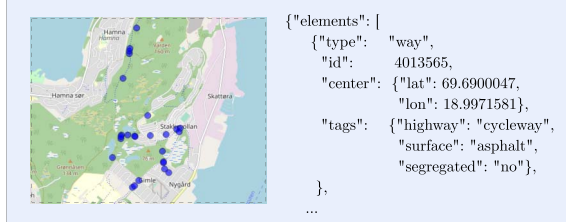


Figure 1: Natural language input and the corresponding Overpass query. The query is executed against the OpenStreetMap database and returns the requested elements in a structured response. The Overpass query language is highly expressive and allows one to formulate complex queries to extract information from OpenStreetMap. Blue tokens in the query are syntax keywords, orange tokens are variable names, and bold tokens define semantic properties of the requested elements. The green token in curly brackets geolocates an area called “Troms”.

Overpass queries from natural language. We finetuned sequence-to-sequence models and found that explicitly pretraining on code is not helpful for the task. We further explored in-context learning strategies for black-box LLMs and found that GPT-4 (OpenAI, 2023) with few-shot examples, retrieved by sentence similarity, yields best results, outperforming the finetuned sequence-to-sequence models.

The proposed Text-to-OverpassQL task, depicted in Figure 1, is a novel semantic parsing problem that is well motivated in real-world applications. While it shares characteristics with the Text-to-SQL task and its accompanying datasets, there are several key differences. The Text-to-OverpassQL task is grounded in a database that is genuinely in use and of global scale. The database is not divided into sub-databases or tables, and each Overpass query can retrieve any of the billions of stored elements in OSM. The desired elements have to be queried by a geographical specification and additional semantic tags. The tags are composed of key-value pairs that follow established community guidelines and conven-

tions, but can also be open-vocabulary. Overall, the proposed task builds on a decades-long effort to structure and store the geographical world around us to make it computationally accessible. With this work, we offer all components to benchmark future semantic parsing systems on a challenging real-world task.

Our main contributions are as follows: (i) We present OverpassNL, a dataset of 8.5k natural language inputs paired with real-world Overpass queries. (ii) We define task-specific evaluation metrics that take the OverpassQL syntax into account and are grounded in database execution. (iii) We train and evaluate several state-of-the-art sequence generation models to establish base performance and to identify specific properties of the proposed Text-to-OverpassQL task.

2 Background

2.1 OpenStreetMap

OpenStreetMap (OSM) is a free and open geographic database that has been created and is maintained by a global community of voluntary contributors. The ever-growing community has over 10M registered members who have collectively contributed to the creation of the existing 9B elements in the database (OpenStreetMap Wiki, 2022). Elements are either nodes, ways, or relations. Nodes are annotated with geospatial coordinates. Ways are composed of multiple nodes and represent roads, building outlines, or area boundaries. Relations describe the relationships of elements, e.g., forming a municipal or major highway. Elements can be tagged with key-value pairs that assign semantic meaning and meta information. The OSM database is widely used in geodata analysis, scientific research, route planning applications, humanitarian aid projects, or augmented reality games. It also serves as a data source for geospatial services of companies like Facebook, Amazon, or Apple (OpenStreetMap Foundation, 2019).

2.2 Overpass Query Language

The Overpass Query Language (OverpassQL) is a “procedural, imperative programming language written with a C style syntax” (OpenStreetMap Wiki, 2023). It is used to query the OpenStreetMap database for geographic data and features. OverpassQL allows for detailed queries that are

capable of extracting elements based on specific criteria, such as certain types of buildings, streets, or natural features within a defined area. Users can specify the types of elements they are interested in, and filter them by their associated key-value pairs.

Query Syntax We briefly explain the OverpassQL syntax based on the query depicted in Figure 1 and refer to the official language guide for more information.² The keyword *geocodeArea* in the first line triggers a geolocation service³ to find an area named “*Troms*”. The retrieved area is then assigned to a variable named *searchArea*. The third line queries for nodes that are tagged with the *natural=peak* key-value pair. The search is limited to nodes that are geographically within the previously defined *searchArea*. The nodes that fulfill these criteria are stored into the *peaks* variable. Line 5 queries for ways within the same area that are tagged with *highway=cycleway* and are within a radius of 500 meters around a node stored in *peaks*. Finally, the query requests a return of the specified ways.

3 Related Work

Natural Language Interfaces for Geodata

One of the first attempts to build a natural language interface for geographical data is GEOQUERY (Zelle and Mooney, 1996; Kate et al., 2005a). It is a system based on Prolog, later adapted to SQL, and is tailored for a small database of U.S. geographical facts. Following work proposed methods to map the text input to the structured query language (Zettlemoyer and Collins, 2005; Kate et al., 2005b). A more recent attempt is NLmaps (Haas and Riezler, 2016; Lawrence and Riezler, 2016), which aimed to build a natural language interface for OpenStreetMap. For querying the database, they designed a machine readable language (MRL). The MRL is an abstraction of the Overpass Query Language, but it supports only a limited number of its features. To facilitate building more potent neural sequence-to-sequence parsers, they later released NLmaps v2 (Lawrence and Riezler, 2018) with augmented text and query pairs. Our work aims to support the Overpass Query Language without simplifications or ab-

stractions, allowing to fully leverage the effort of the OpenStreetMap community that developed a query language that is optimally suited for the large-scale geospatial information in the OSM database.

Text-to-SQL The Text-to-SQL task (Tang and Mooney, 2001; Iyer et al., 2017; Li and Jagadish, 2014; Yaghmazadeh et al., 2017; Zhong et al., 2017) is closely related to the proposed Text-to-OverpassQL task and aims to provide a natural language interface to relational databases. While most of the work in this area focuses on a specific domain and database, the Spider (Yu et al., 2018) dataset provides text and queries for multiple databases spanning different domains. They emphasize the hurdles of collecting real databases with complex schemas and sufficient data records, and circumvent this problem by mainly sourcing the databases from educational material and populate them with synthetic data. In contrast, the OSM database is of global scale and is used in real-world applications. We highlight more differences between the datasets and underlying databases in Section 4.4. The Text-to-SQL task is commonly treated as a sequence-to-sequence problem (Lin et al., 2020; Yu et al., 2021). Some methods explicitly encode the database schema (Zhang et al., 2019), while Scholak et al. (2021) show that finetuning a pretrained T5 model (Raffel et al., 2019) matches the performance of more specialized systems. They further introduced PICARD, a constraint decoding method that is SQL specific and enforces syntactic correctness. With the advent of large language models and in-context learning, more recent work focuses on prompt engineering the task (Sun et al., 2023; Chen et al., 2023; Pourreza and Rafiei, 2023).

4 OverpassNL Dataset

In order to facilitate the Text-to-OverpassQL task, we constructed a parallel dataset of natural language inputs and Overpass queries. This was done by collecting queries written and shared by users of Overpass Turbo,⁴ a web tool that allows users to develop and execute Overpass queries within a graphical interface. We presented the queries to

²https://wiki.openstreetmap.org/wiki/Overpass_API/Language_Guide.

³<https://nominatim.org/>.

⁴The shared queries constitute an unrestricted collection that is publicly available on <https://overpass-turbo.eu/>. The website is maintained by Martin Raifer, who helped us to acquire queries shared between 2014–2022.

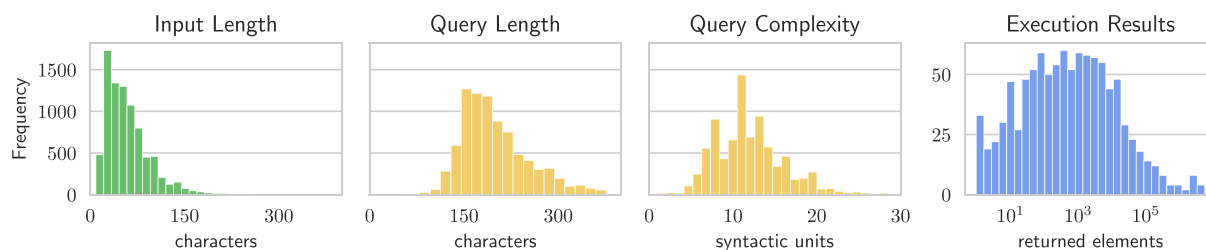


Figure 2: Dataset statistics. Number of elements returned when executing the queries in the development set against OpenStreetMap. Each query returns at least one element and often several orders of magnitude more.

trained annotators who were tasked with writing natural language descriptions for them.

Initiating the dataset creation process with Overpass queries authored by real users and developers has several advantages: Firstly, the queries were created to satisfy legitimate information needs and, as such, cover a wide range of OverpassQL features. Also, the geographical coverage is high, as is shown in Figure 3 by the location distribution of elements returned by executing the queries in our dataset. Another advantage is that it is easier to teach annotators how to interpret Overpass queries than how to write them from scratch.

4.1 Query Annotation

For the annotation task, we recruited university students with proficient English skills and experience in database query languages like SQL. They had to complete a tutorial about OverpassQL and were subsequently tested on their knowledge. The test consisted of multiple choice questions and assignments to write text descriptions of pre-selected Overpass queries. Only the 15 students who passed this test were selected to participate in our annotation task. We built a graphical interface that showed them an Overpass query, the raw execution results, and the results rendered on a map. The task of the annotators was to write a natural language description that best represents the query. To ensure quality, we encouraged the annotators to use the Overpass documentation, continuously conducted spot tests on the submitted inputs, and required the annotators to validate the inputs written by other annotators. We paid 100€ per 250 annotations, resulting in a wage of around 20€/hour.

4.2 Dataset Statistics

In total we obtained 8,352 queries annotated with natural language inputs. We split these into 6,352

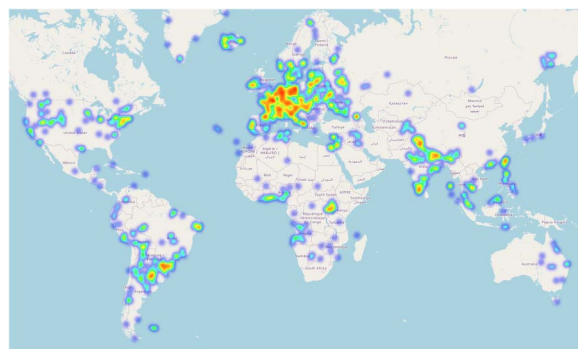


Figure 3: Location distribution of results returned by Overpass queries in our dataset. The queries cover locations on all continents. Europe is a traditional hotspot of the OpenStreetMap community and also has the best mapping coverage.

instances for training, 1,000 for development, and 1,000 test instances. We constructed the splits such that there are no (near) duplicates on the input side or query side between training and evaluation instances. Figure 2 gives some statistics illustrating important dataset properties. There are a total of 11,259 distinct words in the natural language inputs, the mean input length is 59.7 characters, the mean query length is 199.8 characters, and each query has an average of 11.9 syntactic units. A syntactic unit is a subtree in the XML representation of an Overpass query. The rightmost plot in Figure 2 shows the number of elements returned by executing the development set queries.

4.3 Complexity & Coverage

There are a total of 41 major syntax features⁵ in the Overpass Query Language. Thirty-one of these 41 features occur in at least 20 queries of our dataset, resulting in a feature coverage rate of 76%. See Figure 4 for a partial list of syntax features. Our queries utilize 1,046 unique keys to

⁵https://wiki.openstreetmap.org/wiki/Overpass_API/Overpass.QL.

- Settings (5 of 6 features used in our dataset)
 - Date (286; 3.4%)
 - Diff/adiff two dates (0; 0%)
 - ...
- Block Statements (4 of 8)
 - Union (7,651; 91.6%)
 - Difference (231; 2.8%)
 - if (5; 0.1%)
 - for (85; 1.0%)
 - ...
- Standalone Statements (9 of 13)
 - Recurse Up (30; 0.4%)
 - Recurse Down (6,084; 72.8%)
 - is_in (22; 0.3%)
 - The Query Statement (8,313; 99.5%)
 - The Query Filter (120; 1.4%)
 - ...
- Filters (12 of 14)
 - By Tag (7,860; 94.1%)
 - Bounding Box (2,548; 30.5%)
 - By Element Id (3,571; 42.8%)
 - Newer (286; 3.4%)
 - ...

Figure 4: Partial list of Overpass Query Language syntax features with absolute and relative occurrence in the 8,352 dataset queries.

specify tagged elements. These keys cover 91% of all key usage in OpenStreetMap. The coverage of corresponding values is harder to estimate because of open-class keys like *name*, *source*, or *operator*. There are also keys that have recommended sets of values, e.g., the key *leisure* is commonly paired with *swimming_pool*, *skatepark*, or *pitch*. In total, we count 3,879 unique values and 4,880 unique key-value pairs in our dataset queries.

4.4 Comparison to Other Datasets

Because we are the first to present a dataset for the Text-to-OverpassQL task, we compare it to datasets of related tasks (see Table 1). GEOQUERY (Zelle and Mooney, 1996) is a small-scale dataset with 880 instances that allow one to query 937 different geographical facts about the United States. NLmaps (Haas and Riezler, 2016) and NLmap v2 (Lawrence and Riezler, 2018) provide queries for OpenStreetMap paired with natural language inputs, however, the queries are written in their own restricted query language called MRL. In contrast,

we generate queries in the well-established OverpassQL language that has more features and is widely used by the OpenStreetMap community. Additionally, the NLmaps datasets only include up to 347 distinct keys in key-value pairs, limiting the semantic expressiveness of generated queries. Furthermore, NLmaps was built for an older version of the OSM database comprising one third of the size of the current version used in our work. WikiSQL (Zhong et al., 2017) converts single tables from Wikipedia articles to SQL databases and annotates queries with natural language inputs for them. While the dataset is of large scale, it contains only simple SQL queries and unconnected tables. The Spider dataset (Yu et al., 2018) includes 166 distinct databases of different domains and was collected for the Text-to-SQL task. Each natural language input in the dataset is intended for a specific database that is known a priori. This significantly reduces the number of relevant table names and column names per query, and simplifies the task by allowing to append known table and column names together with the database schema to the input. This stands in stark contrast to our Text-to-OverpassQL task where each query can utilize any key-value pair and retrieve elements from the entire OSM database, making it harder to predict the correct named identifiers for the desired elements. Also, the number of returned elements per query is orders of magnitude larger than in SQL related datasets. This makes the grounded evaluation harder by minimizing the likelihood of false positive matches in execution accuracy.

5 Task & Evaluation

The Text-to-OverpassQL task requires a user to generate an Overpass query *q*, given a natural language input *x*. A model for this task aims to accurately translate the request, formulated in natural language, into code, written in the Overpass Query Language, that returns the correct elements when executed against the OpenStreetMap database. In order to evaluate such a system, we propose to use different evaluation metrics. These include metrics that compare the generated query with the reference query, and metrics that compare the results returned by executing the queries against the OSM database. In order to make the execution results reproducible, we release the evaluation script and a Docker container

Dataset	Query Language	Number of Instances	Query Templates	Named Identifiers		Extractable Elements		Results
				total	per database	total	per database	per query
GEOQUERY	SQL	880	234	31 & 7	31 & 7	937	51	5
NLmaps	MRL	2,380	379	107	107	3.4B	3.4B	–
NLmaps v2	MRL	28,609	360	347	347	3.4B	3.4B	–
WikiSQL	SQL	81,654	–	168k	6.38	460k	11	1
Spider	SQL	10,181	5,693	4,669 & 876	58 & 5	1.6M	9.6k	30
OverpassNL	OverpassQL	8,352	3,890	1,046	1,046	9B	9B	10k

Table 1: Comparison of Text-to-Query datasets. Templates are normalized queries, i.e., removing named identifiers, variable names and digits. Named identifiers are tag keys in OpenStreetMap related datasets and table & column names in SQL related datasets. The Spider dataset includes 166 unconnected databases and the task is to generate a query for a specific database that is known a priori. Thus, the average number of relevant named identifiers and extractable elements is much smaller per query than for the whole dataset.

with the exact snapshot of the OSM database we used. We further describe the metrics in detail.

5.1 Overpass Query Similarity Evaluation

We propose a language-specific metric called Overpass Query Similarity (OQS) to quantify the compatibility of a generated query and the reference query. The metric is composed of three parts. First, we use character F-score (chrF), which measures the overlap of character n-grams between two strings (Popović, 2015). Because chrF operates on the character-level, it is well suited for Overpass queries which consist of words and special characters alike. Next, we calculate the overlap of keys and values between the generated query q_G and reference query q_R . We define the Key Value Similarity (KVS) as follows:

$$\text{KVS}(\mathbf{q}_G, \mathbf{q}_R) = \frac{|\text{KV}(\mathbf{q}_G) \cap \text{KV}(\mathbf{q}_R)|}{\max(|\text{KV}(\mathbf{q}_G)|, |\text{KV}(\mathbf{q}_R)|)}, \quad (1)$$

where the operator $\text{KV}(\cdot)$ returns all key-value pairs as well as all individual keys and individual values. This metric captures the semantic relatedness of two queries. Complementarily, the third part of the OQS metric compares queries on the syntactic level. We compute the Tree Similarity metric (TreeS) by comparing the XML tree representation of the queries. We remove all key-value pairs and variable names from the trees and recursively compute the number of matching subtrees of the generated and reference query. Analogous to Equation 1, we normalize the number of

matching subtrees by the maximum number of subtrees in either tree. Finally, the proposed Overpass Query Similarity metric is the mean of chrF, KVS and TreeS. The metric captures similarity of system outputs and reference queries on the levels of surface string, semantics, and syntax.

5.2 Grounded Evaluation

The nature of the Text-to-OverpassQL task allows us to perform a grounded evaluation of generated queries by executing them against the OSM database. We quantify the correctness of the database execution by Execution Accuracy (EX), which measures the exact match of all elements returned by executing the generated query and the reference query. Each returned element has an identifier number that is unique within OSM. We use this identifier number to determine exact matching of results. The plot on the right in Figure 2 shows that there are up to 10^7 elements returned by a query. The matching by unique identifier and large number of returned elements make EX an inherently hard metric to satisfy.

Because OpenStreetMap is a community driven database that has grown over decades with changing annotation guidelines, there can be ambiguities in the tags of elements. For example, filtering all bridges can mean `node["bridge"]` or `node["bridge"="yes"]`. Both are correct according to current annotation guidelines, but do return slightly different sets of nodes. Another example is filtering for radar towers:

`node["man_made"="tower"] ["tower:type"="radar"]`.

The first filter is redundant according to tagging guidelines, but cannot be omitted in queries because the guidelines are not consistently followed throughout the whole database. To account for this, we also report Soft Execution Accuracy (EX_{SOFT}). It is computed as the overlap of returned elements, normalized by the maximum number of elements returned by either the generated or reference query. The metric ranges from 0, which means no overlap, to 1, which is equivalent to exact match of results. We report the metric as percent in the results tables.

6 Experiments

In the following, we present experiments that showcase the opportunities to train machine learning models on the OverpassNL dataset and establish base performance of commonly used techniques. We finetuned sequence-to-sequence models of different sizes and pretraining settings. Additionally, we adapted black-box large language models with different in-context learning strategies. All models are evaluated with the proposed similarity and grounded metrics, as well as exact string match (EM).

6.1 Finetuning

The task of generating Overpass queries from text is a sequence-to-sequence problem and can be addressed by a model with encoder-decoder architecture. The encoder processes the input text and the decoder autoregressively generates the Overpass query. In order to choose a suitable pre-trained model, we focused on the T5 family of models because of their strong performance in a variety of sequence-to-sequence tasks (Raffel et al., 2019), in particular Text-to-SQL (Scholak et al., 2021). Besides the vanilla T5 model,⁶ the family also includes models specifically trained for code generation (CodeT5; Wang et al., 2021) and models with byte-level tokenization (ByT5; Xue et al., 2022). Because neither model has been trained on data covering OverpassQL syntax, we ran experiments to compare the finetuning of CodeT5 and ByT5 on the training portion of our dataset. To expand the training set, we additionally created instances from comments that query

authors to put in some lines with the intention to describe the line’s purpose and functionality. We extracted the comments and used them as the natural language input for the respective query. This produced 6,000 additional training instances. We finetune all parameters (full finetuning) for 30 epochs using the Adam (Kingma and Ba, 2015) optimizer with weight decay of 0.1. The maximum learning rate is 4×10^{-4} with warmup for 10% of the steps and a linear decay schedule. Training batch size is 16 and we decode with four search beams.

The upper half of Table 2 shows the results for combinations of model type, model size, and training data, evaluated on the development set. In general, the *base* variant of the models is better than the *small* variant. We did not gain further improvements by finetuning even larger variants of the models. The results also show that the ByT5 models are better suited for our task than the CodeT5 models. Although the instances derived from developer comments are of low quality, they contribute to consistent improvements in execution accuracy for all models. The best model for our task is *ByT5-base* with 582M parameters, finetuned on the enhanced training set. We further refer to this model as *OverpassT5*.

6.2 In-Context Learning

We furthermore explore the use of in-context learning for the Text-to-OverpassQL task. We prompt large language models to generate an Overpass query for the given text input while providing five example pairs as context. The example pairs are selected from the training set, either randomly or by input similarity (Liu et al., 2022). We compare BLEU (Papineni et al., 2002) and sentence-BERT embedding similarity (Reimers and Gurevych, 2019) as metrics to retrieve the most similar examples.

Figure 5 shows that the quality of queries generated by LLaMa (Touvron et al., 2023) increases with the model size. The lower half of Table 2 shows results for the even bigger GPT-3 (*text-davinci-003*; Brown et al., 2020) and GPT-4 (*gpt-4-0314*; OpenAI, 2023) models. We see a similar trend of improved results with increasing model size. Furthermore, retrieving similar instances from the training set as in-context examples is consistently better than a random selection. We also see that retrieval by sentence-BERT

⁶The vanilla T5 model is not suited for code or Overpass because it lacks tokens for ‘{’ or ‘[’ (Wang et al., 2021).

Model	Setting	Overpass Query Similarity				EM	Execution Accuracy	
		chrF	KVS	TreeS	OQS		EX	EX _{SOFT}
Finetuning								
CodeT5-small		74.0 $\pm$ 0.2	61.2 $\pm$ 0.6	72.2 $\pm$ 0.1	69.1 $\pm$ 0.2	18.5 $\pm$ 0.1	31.9 $\pm$ 0.9	41.9 $\pm$ 0.6
CodeT5-small	+comments	74.1 $\pm$ 0.2	62.4 $\pm$ 0.7	72.7 $\pm$ 0.3	69.8 $\pm$ 0.2	18.9 $\pm$ 0.4	32.7 $\pm$ 0.6	43.9 $\pm$ 0.5
CodeT5-base		74.6 $\pm$ 0.0	63.2 $\pm$ 0.4	73.0 $\pm$ 0.1	70.3 $\pm$ 0.2	19.8 $\pm$ 0.4	33.3 $\pm$ 0.3	44.3 $\pm$ 0.1
CodeT5-base	+comments	74.9 $\pm$ 0.1	63.6 $\pm$ 0.5	73.5 $\pm$ 0.1	70.7 $\pm$ 0.2	20.3 $\pm$ 0.2	34.5 $\pm$ 0.4	46.2 $\pm$ 0.4
ByT5-small		74.8 $\pm$ 0.2	64.4 $\pm$ 0.2	73.1 $\pm$ 0.2	70.8 $\pm$ 0.2	20.4 $\pm$ 0.1	35.4 $\pm$ 0.3	46.8 $\pm$ 0.3
ByT5-small	+comments	75.0 $\pm$ 0.0	64.6 $\pm$ 0.2	73.4 $\pm$ 0.2	71.0 $\pm$ 0.1	21.0 $\pm$ 0.3	36.0 $\pm$ 0.4	46.2 $\pm$ 0.5
ByT5-base		75.5 $\pm$ 0.0	65.0 $\pm$ 0.2	73.8 $\pm$ 0.2	71.4 $\pm$ 0.0	21.9 $\pm$ 0.4	36.2 $\pm$ 0.0	46.7 $\pm$ 0.4
ByT5-base	+comments	75.5 $\pm$ 0.1	66.0 $\pm$ 0.2	73.7 $\pm$ 0.4	71.7 $\pm$ 0.1	22.0 $\pm$ 0.1	36.7 $\pm$ 0.6	47.0 $\pm$ 1.0
5-Shot In-Context Learning								
GPT-3	random	58.8	48.8	57.5	55.0	4.1	16.9	28.0
GPT-3	retrieval-BLEU	67.4	55.5	66.8	63.3	18.0	28.7	37.7
GPT-3	retrieval-sBERT	72.1	63.3	69.9	68.4	19.5	34.1	44.2
GPT-4	random	63.8	57.5	61.1	60.8	5.1	25.4	39.5
GPT-4	retrieval-BLEU	74.3	66.1	72.4	71.0	22.9	38.5	50.7
GPT-4	retrieval-sBERT	75.7	69.9	74.0	73.2	23.4	40.4	53.0

Table 2: Results on the **development set** of OverpassNL. The proposed Overpass Query Similarity (OQS) metric is the mean of Character F-score (chrF), Key-Value similarity (KVS), and XML-tree similarity (TreeS). EM denotes exact string match. Execution accuracy (EX) is the exact match of all results returned by executing the generated query and reference query against OpenStreetMap. Soft Execution Accuracy (EX_{SOFT}) is the normalized overlap of returned results. Results in **bold** are best for the respective learning setup. Finetuning experiments are repeated three times with different random seeds and mean/standard deviation are reported.

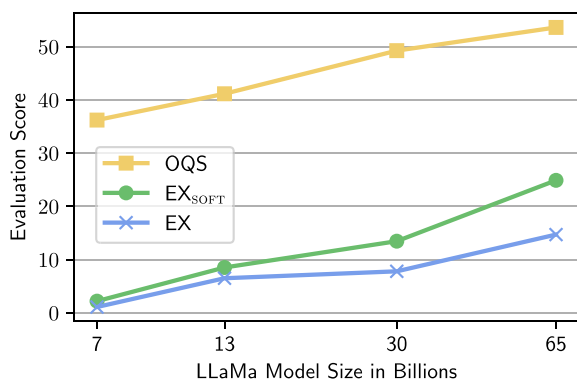


Figure 5: Development set results of LLaMa models with increasing number of parameters, prompted with five in-context examples.

embedding similarity is better than retrieval by BLEU score. Best results are obtained for GPT-4 with sBERT retrieval, which will simply be referred to as *GPT-4* in the following.

6.3 Comparison between Finetuning and In-Context Learning

In the previous sections, we selected the best models for finetuning and for in-context learning, based on development set performance. In order to compare the two models, we present results on the test set in Table 3 (top). While the surface metrics OQS and EM are nearly identical for both models, GPT-4 significantly outperforms OverpassT5 in the execution based metrics. It is also interesting that the queries generated by GPT-4 are more likely to raise syntax errors (#Errors), despite achieving higher execution accuracy. Inspecting the individual components of OQS reveals that GPT-4 is better at generating correct key-value pairs, indicating that they are more important for correct execution results than faithfulness to the syntax of the reference query. We conjecture that the reason is that GPT-4 has likely seen a larger amount of OSM key-value pairs

Model	Overpass Query Similarity				#Errors	Execution Accuracy		
	chrF	KVS	TreeS	OQS		EX	EX _{SOFT}	
Full Test Set (1,000 Instances)								
OverpassT5	74.9 ±0.1	66.1 ±0.3	72.7 ±0.2	71.2 ±0.2	20.7 ±0.2	23 ±5.6	33.9 ±0.1	46.3 ±0.3
GPT-4	73.6	68.6	72.0	71.4	20.7	34	38.9	53.0
Hard Partition (333 Instances)								
OverpassT5	62.8 ±0.4	57.7 ±0.4	56.6 ±0.3	59.1 ±0.3	8.8 ±0.3	15 ±4.5	18.7 ±0.1	29.7 ±0.5
GPT-4	61.4	59.6	56.3	59.1	9.0	25	22.2	35.3

Table 3: Results on the **test set** of our OverpassNL dataset. The proposed Overpass Query Similarity (OQS) metric is the mean of Character F-score (chrF), Key-Value Similarity (KVS), and XML-tree Similarity (TreeS). Exact match (EM) is query string match. #Errors are number of raised syntax errors when trying to execute the query. Execution accuracy (EX) is the exact match of all results returned by executing the model generated query and reference query against OpenStreetMap. Soft Execution Accuracy (EX_{SOFT}) is the normalized overlap of returned results. **Bold** results are best on the test set and underlined results are best on the hard partition.

during pretraining than the finetuned models that are limited to OSM knowledge acquired from our training set. Table 3 (bottom) shows the results on the hard partition of the test set (defined in Section 7.1). All performance metrics decrease significantly, without affecting the relative improvement of GPT-4 over OverpassT5. Notably, the majority of syntax errors stem from this partition of the test set.

In sum, while GPT-4 is better at generating queries that return correct results, the OverpassT5 model is better at producing faithful OverpassQL syntax. However, GPT-4’s advantage in execution accuracy comes at a computational cost. OverpassT5 is orders of magnitude smaller, resulting in faster inference speed and likely lower monetary cost per query. Also, GPT-4 is a 3rd-party API and does not allow for self-hosting, implying privacy concerns.

7 Analysis

7.1 Instance Difficulty

To better assess the performance of our proposed models, we aim to divide the evaluation instances into three difficulty partitions. Figure 6 displays EX_{SOFT} results for the easy, medium, and hard partitions according to different difficulty criteria. A straightforward difficulty metric like the length of the input text has little influence on the

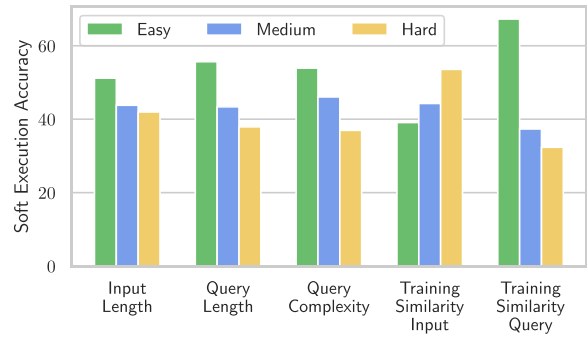


Figure 6: Instance difficulty on the development set using OverpassT5. Dividing the evaluation instances by highest similarity to any training query allows to measure performance on instances with different difficulties.

accuracy. Using the length or complexity of the query (measured as the number of syntactic units) as difficulty metric leads to a clearer partition into easy, medium, and hard instances. Surprisingly, partitioning the instances based on their maximum input text similarity to any training instance leads to an undesired negative correlation of EX_{SOFT} and difficulty where the instances with the lowest similarity to training inputs achieve best EX_{SOFT} results. Finally, the clearest partition into instances of different difficulty is achieved by using the maximum similarity of a query to any query in the training set, where query similarity is measured by the OQS metric. A partitioning based on similarity to training queries can be seen

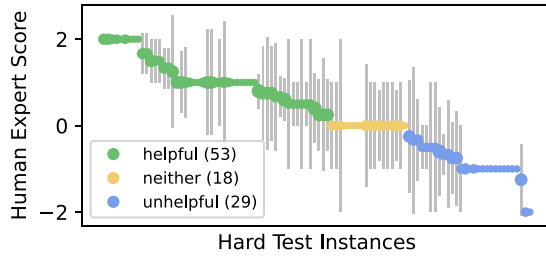


Figure 7: Human expert evaluation for 100 hard instances of the test set. Dot size indicates the number of human evaluators (up to 5 per instance). Gray bars depict the standard deviation.

as *out-of-distribution* testing since it measures the performance for instances that are less likely to be memorized from the training set. We thus use this criterion to select instances constituting the hard partition in the experiments in Section 6.

7.2 Human Expert Evaluation

One motivation of the Text-to-OverpassQL task is to facilitate a system that assists expert users with crafting queries. While the surface and execution metrics allow us to compare different models, it is difficult to estimate how helpful imperfect queries are to human developers. To this end, we conducted an evaluation of the OverpassT5 outputs by human experts. They are Overpass developers that were acquired by postings in Overpass communities and they participated voluntarily in our experiment. They had to rate the helpfulness of a query given the input text on a scale from “very unhelpful” to “very helpful”. There were seven experts casting a total of 228 votes across 100 instances of the hard partition. The results in Figure 7 show that the majority of generated queries were rated helpful, and less than a third were deemed unhelpful. This shows that even for the hardest test instances, the OverpassT5 model generates queries that are mostly helpful for developers when crafting a new query.

7.3 Self-Refinement from Execution Feedback

Recently, studies have shown that LLMs are able to self-refine their own outputs (Madaan et al., 2023). The generated hypothesis is appended to the context and the LLM is prompted to generate an improved version. We conduct self-refine experiments for GPT-4 by appending the generated query and by additionally providing feedback

Model	OQS	EM	#Errors	EX	EX _{SOFT}
GPT-4	73.2	23.4	24	40.4	53.0
Refine Syntax Errors Only					
no feedback	73.2	23.4	19	40.4	53.1
with feedback	73.2	23.4	7	40.7	53.7
Refine All Instances					
no feedback	73.1	21.9	31	39.6	52.9
with feedback	73.1	23.4	26	41.4	54.5

Table 4: Self-Refinement of hypotheses generated by GPT-4 on the **development set**. Feedback is either the error message during execution or the returned results.

The OverpassQL language allows one to formulate questions to the OpenStreetMap database. Your goal is, given an Input and a Hypothesis, to produce a improved version of the Hypothesis. If the Hypothesis is already good enough, do not try to improve it. Here are a few examples:

Input: *text_input_1*
Hypothesis: *overpass_query_hypothesis_1*
Overpass Query: *overpass_query_1*

... four more in-context examples ...

Input: **unseen_text_input**

Here is the Overpass Query Hypothesis produced by a model: **overpass_query_hypothesis**

You will now get part of the Overpass result produced after using the generated Overpass Query Hypothesis. An error means that you should definitely improve on the Hypothesis. A normal result could mean that the Overpass Query is good enough, if the output fits to the asked query: **execution_feedback**

Improve on the Overpass Query or keep it if it is good enough: <>

Figure 8: Prompt for 5-shot refine with feedback.

from the query execution. If the query cannot be executed, we use the error message as feedback, otherwise we append a sample of the returned elements to the prompt. Table 4 shows results for self-refinement in two scenarios where either self-refinement is applied to all queries, or only to queries that raised a syntax error during execution. The results show that only refining

syntax errors reduces the error count from 24 to 7 if an explicit error message is appended to the prompt, compared to a reduction to 19 errors if only the generated query is appended. However, this only leads to a slight increase in execution accuracy. On the other hand, refining all instances leads to an increase in errors, but also improves EX_{SOFT} by 1.5 points when providing explicit feedback. These experiments show that there is still room for improving query generation with clever prompting techniques. Figure 8 shows the prompt incorporating hypothesis and execution feedback. The feedback is either an error if raised during execution, the returned results, or “No results found”.

8 Conclusion

We introduced a novel semantic parsing task, called Text-to-OverpassQL. The objective of this task is to generate Overpass queries from natural language inputs. We highlighted its relevance within the OpenStreetMap ecosystem and related it to similar tasks. We identified key differences to the Text-to-SQL that pose unique challenges. To facilitate research on the task, we proposed OverpassNL, a dataset of real-world Overpass queries and corresponding annotations with natural language inputs. We used this dataset to train several state-of-the-art models and establish a base performance of the task. In order to measure prediction performance, we proposed task-specific metrics that take the OverpassQL syntax into account and are grounded in database execution. We presented a detailed evaluation of results that reveals the strengths and weaknesses of the considered learning strategies. We hope that our work serves as a foundation for further research on the challenging task of semantic parsing of geographical information needs grounded in the large and widely used OpenStreetMap database.

Acknowledgments

The research reported in this paper was supported by a Google Focused Research Award on “Learning to Negotiate Answers in Multi-Pass Semantic Parsing”. We also thank Martin Raifer for helping us to acquire queries shared by Overpass Turbo users.

References

- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc.
- Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching large language models to self-debug. *arXiv preprint arXiv:2304.05128*.
- Carolin Haas and Stefan Riezler. 2016. A corpus and semantic parser for multilingual natural language querying of OpenStreetMap. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 740–750, San Diego, California. Association for Computational Linguistics. <https://doi.org/10.18653/v1/N16-1088>
- Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, Jayant Krishnamurthy, and Luke Zettlemoyer. 2017. Learning a neural semantic parser from user feedback. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 963–973, Vancouver, Canada. Association for Computational Linguistics. <https://doi.org/10.18653/v1/P17-1089>
- Rohit J. Kate, Yuk Wah Wong, and Raymond J. Mooney. 2005a. Learning to transform natural to formal languages. In *Proceedings of the 20th National Conference on Artificial Intelligence - Volume 3, AAAI’05*, pages 1062–1068. AAAI Press.
- Rohit J. Kate, Yuk Wah Wong, and Raymond J. Mooney. 2005b. Learning to transform natural to formal languages. In *Proceedings of*

- the *Twentieth National Conference on Artificial Intelligence (AAAI-05)*, pages 1062–1068, Pittsburgh, PA.
- Diederik P. Kingma and Jimmy Ba. 2015. Adam: A method for stochastic optimization. In *Proceedings of the International Conference on Learning Representations (ICLR)*. San Diego, California.
- Carolyn Lawrence and Stefan Riezler. 2016. NLmaps: A natural language interface to query OpenStreetMap. In *Proceedings of COLING 2016, the 26th International Conference on Computational Linguistics: System Demonstrations*, pages 6–10, Osaka, Japan. The COLING 2016 Organizing Committee.
- Carolyn Lawrence and Stefan Riezler. 2018. Improving a neural semantic parser by counterfactual learning from human bandit feedback. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1820–1830, Melbourne, Australia. Association for Computational Linguistics. <https://doi.org/10.18653/v1/P18-1169>
- Fei Li and H. V. Jagadish. 2014. Constructing an interactive natural language interface for relational databases. *Proceedings of the VLDB Endowment*, 8(1):73–84. <https://doi.org/10.14778/2735461.2735468>
- Xi Victoria Lin, Richard Socher, and Caiming Xiong. 2020. Bridging textual and tabular data for cross-domain text-to-sql semantic parsing. *CoRR*, abs/2012.12627. <https://doi.org/10.48550/arXiv.2012.12627>
- Jiachang Liu, Dinghan Shen, Yizhe Zhang, Bill Dolan, Lawrence Carin, and Weizhu Chen. 2022. What makes good in-context examples for GPT-3? In *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd Workshop on Knowledge Extraction and Integration for Deep Learning Architectures*, pages 100–114, Dublin, Ireland and Online. Association for Computational Linguistics. <https://doi.org/10.18653/v1/W15-3049>
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*.
- OpenAI. 2023. Gpt-4 technical report. *ArXiv*, abs/2303.08774.
- OpenStreetMap Foundation. 2019. Who uses openstreetmap? | openstreetmap. [Online; accessed 24-July-2023].
- OpenStreetMap Wiki. 2022. Stats—openstreetmap wiki. [Online; accessed 24-July-2023].
- OpenStreetMap Wiki. 2023. Overpass api/overpass ql—openstreetmap wiki. [Online; accessed 24-July-2023].
- Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. BLEU: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pages 311–318, Philadelphia, Pennsylvania, USA. Association for Computational Linguistics. <https://doi.org/10.3115/1073083.1073135>
- Maja Popović. 2015. chrF: Character n-gram F-score for automatic MT evaluation. In *Proceedings of the Tenth Workshop on Statistical Machine Translation*, pages 392–395, Lisbon, Portugal. Association for Computational Linguistics. <https://doi.org/10.18653/v1/W15-3049>
- Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Colin Raffel, Noam M. Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2019. Exploring the limits of transfer learning with a unified text-to-text transformer. *ArXiv*, abs/1910.10683.
- Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*,

- pages 3982–3992, Hong Kong, China. Association for Computational Linguistics. <https://doi.org/10.18653/v1/D19-1410>
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 9895–9901, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics. <https://doi.org/10.18653/v1/2021.emnlp-main.779>
- Ruoxi Sun, Sercan O. Arik, Hootan Nakhost, Hanjun Dai, Rajarishi Sinha, Pengcheng Yin, and Tomas Pfister. 2023. Sql-palm: Improved large language model adaptation for text-to-sql. *arXiv preprint arXiv:2306.00739*.
- Lappoon R. Tang and Raymond J. Mooney. 2001. Using multiple clause constructors in inductive logic programming for semantic parsing. In *Machine Learning: ECML 2001*, pages 466–477, Berlin, Heidelberg. Springer Berlin Heidelberg. https://doi.org/10.1007/3-540-44795-4_40
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. Llama: Open and efficient foundation language models. <https://doi.org/10.48550/arXiv.2302.13971>
- Yue Wang, Weishi Wang, Shafiq Joty, and Steven C. H. Hoi. 2021. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, EMNLP 2021*. <https://doi.org/10.18653/v1/2021.emnlp-main.685>
- Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. 2022. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 10:291–306. <https://doi.org/10.1162/tacl.a-00461>
- Navid Yaghmazadeh, Yuepeng Wang, Isil Dillig, and Thomas Dillig. 2017. Sqlizer: Query synthesis from natural language. *Proceedings of the ACM on Programming Languages*, 1(OOPSLA). <https://doi.org/10.1145/3133887>
- Tao Yu, Chien-Sheng Wu, Xi Victoria Lin, bailin wang, Yi Chern Tan, Xinyi Yang, Dragomir Radev, Richard Socher, and Caiming Xiong. 2021. Gra{pp}a: Grammar-augmented pre-training for table semantic parsing. In *International Conference on Learning Representations*.
- Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Brussels, Belgium. <https://doi.org/10.18653/v1/D18-1425>
- John M. Zelle and Raymond J. Mooney. 1996. Learning to parse database queries using inductive logic programming. In *Proceedings of AAAI/IAAI*, pages 1050–1055, Portland, OR. AAAI Press/MIT Press.
- Luke S. Zettlemoyer and Michael Collins. 2005. Learning to map sentences to logical form: Structured classification with probabilistic categorical grammars. In *Proceedings of the Twenty-First Conference on Uncertainty in Artificial Intelligence, UAI’05*, pages 658–666, Arlington, Virginia, USA. AUAI Press.
- Rui Zhang, Tao Yu, Heyang Er, Sungrok Shim, Eric Xue, Xi Victoria Lin, Tianze Shi, Caiming Xiong, Richard Socher, and Dragomir Radev. 2019. Editing-based SQL query generation for cross-domain context-dependent questions. In

Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), Hong Kong, China. <https://doi.org/10.18653/v1/D19-1537>

Victor Zhong, Caiming Xiong, and Richard Socher. 2017. Seq2sql: Generating structured queries from natural language using reinforcement learning. *CoRR*, abs/1709.00103. <https://doi.org/10.48550/arXiv.1709.00103>